# Assignment No: 13

**Title:** Develop a PHP-MySQL application to perform database connectivity and CRUD operations.

---

**Aim:** To develop a PHP application that connects to a MySQL database and performs the four CRUD operations - Create, Read, Update and Delete - on a manuscript archive table, using prepared statements to prevent SQL injection.

---

**Objectives:**

1. To design a relational table and create it with SQL DDL.
2. To connect PHP to MySQL using the MySQLi extension and handle connection errors.
3. To insert a new record (Create) from a validated HTML form.
4. To retrieve and display records in an HTML table (Read), including a search filter.
5. To update an existing record identified by its primary key (Update).
6. To delete a record safely with a confirmation step (Delete).
7. To use prepared statements with bound parameters to prevent SQL injection.
8. To understand the difference between MySQLi and PDO.

---

**Hardware Used:**

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

**Software Used:**

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- XAMPP / WAMP / LAMP stack (Apache + PHP 8 + MySQL / MariaDB)
- phpMyAdmin (for inspecting the database)

---

**Theory:**

**1. Databases and CRUD**

A relational database stores data in tables made of rows and columns. CRUD names the four basic persistent-storage operations and their SQL statements.

| Operation | SQL statement | Typical HTTP method |
|---|---|---|
| Create | INSERT INTO | POST |
| Read | SELECT | GET |
| Update | UPDATE ... WHERE | POST / PUT |
| Delete | DELETE FROM ... WHERE | POST / DELETE |

## 2. Connecting PHP to MySQL

PHP offers two modern interfaces: MySQLi, which works with MySQL only, and PDO, which works with twelve database systems through one API. Both support prepared statements. The old mysql_* functions were removed in PHP 7 and must not be used.

### MySQLi connection:

```
$conn = new mysqli("localhost", "root", "", "lipisutra");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
$conn->set_charset("utf8mb4");   // required for Devanagari text
```

### PDO connection:

```
$pdo = new PDO("mysql:host=localhost;dbname=lipisutra;charset=utf8mb4",
               "root", "",
               [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
```

## 3. SQL Injection and Prepared Statements

Concatenating user input into an SQL string lets an attacker change the meaning of the query. If $id comes from the URL, the query "DELETE FROM manuscripts WHERE id = $id" with $id set to "1 OR 1=1" deletes the whole table.

A prepared statement sends the SQL template and the values separately, so a value can never be interpreted as SQL. The template uses ? placeholders, and bind_param declares the type of each one: i for integer, d for double, s for string, b for blob.

### Safe pattern:

```
$stmt = $conn->prepare("DELETE FROM manuscripts WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$stmt->close();
```

## 4. Reading a Result Set

execute() runs the statement; get_result() returns a mysqli_result for a SELECT. fetch_assoc() returns the next row as an associative array, or null when the rows are exhausted, so it is normally driven by a while loop. num_rows gives the row count and affected_rows gives the number of rows changed by an INSERT, UPDATE or DELETE.

## 5. Table Design

| Column | Type | Purpose |
|---|---|---|
| id | INT AUTO_INCREMENT PRIMARY KEY | Surrogate key |
| accession | VARCHAR(12) UNIQUE NOT NULL | Museum accession number |
| title | VARCHAR(120) NOT NULL | Human-readable title |
| script | ENUM(...) NOT NULL | Detected script |
| era_year | INT | Estimated year, negative for BCE |
| confidence | DECIMAL(5,2) | OCR confidence percentage |

| | | |
|---|---|---|
| status | ENUM('pending','verified') | Review state |
| transcript | TEXT | Decoded text (utf8mb4 for Devanagari) |
| created_at | TIMESTAMP DEFAULT CURRENT_TIMESTAMP | Insertion time |

---

## Course Project Implementation:

**Course-Project Mapping:** Not available in the course project as PHP-MySQL. LipiSutra has no PHP and no relational database. It does, however, implement the full CRUD cycle against a NoSQL document store: src/services/firebase.js performs Create with setDoc and addDoc, Read with getDoc and getDocs, Update with updateDoc, and src/components/HistorianView.jsx performs a batched update through writeBatch. That file is the project's conceptual equivalent and is reproduced below as supporting evidence, but it cannot be submitted as the implementation because the assignment specifically requires PHP and MySQL. A complete PHP-MySQL application is therefore supplied.

| CRUD operation | PHP-MySQL (supplied) | LipiSutra equivalent (NoSQL) |
|---|---|---|
| Create | INSERT INTO manuscripts | setDoc() / addDoc() in firebase.js |
| Read | SELECT ... WHERE | getDoc() / getDocs() with query + orderBy + limit |
| Update | UPDATE ... WHERE id = ? | updateDoc() and writeBatch() in HistorianView |
| Delete | DELETE FROM ... WHERE id = ? | not implemented in the project |

## Program:

### File 1: lipisutra.sql - database and table creation

```
CREATE DATABASE IF NOT EXISTS lipisutra
  DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE lipisutra;

CREATE TABLE IF NOT EXISTS manuscripts (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  accession   VARCHAR(12)  NOT NULL UNIQUE,
  title       VARCHAR(120) NOT NULL,
  script      ENUM('Modi','Devanagari','Brahmi','Sharada') NOT NULL,
  era_year    INT          NOT NULL,
  confidence  DECIMAL(5,2) NOT NULL DEFAULT 0.00,
  status      ENUM('pending','verified') NOT NULL DEFAULT 'pending',
  transcript  TEXT,
  created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO manuscripts (accession, title, script, era_year, confidence, status,
transcript)
VALUES
 ('LS-0101','Modi Land Grant','Modi',1712,92.50,'verified','...'),
```

```
 ('LS-0102','Devanagari Verse Folio','Devanagari',1588,87.00,'verified','...'),
 ('LS-0103','Brahmi Rock Inscription','Brahmi',-230,71.25,'pending','...');
```

### File 2: db.php - the shared connection

```
<?php
$HOST = "localhost";
$USER = "root";
$PASS = "";
$NAME = "lipisutra";

$conn = new mysqli($HOST, $USER, $PASS, $NAME);

if ($conn->connect_error) {
    die("Database connection failed: " . $conn->connect_error);
}

// utf8mb4 is required so Devanagari and Modi transcripts are stored correctly
$conn->set_charset("utf8mb4");
```

### File 3: index.php - READ with search, plus the DELETE link

```
<?php
require "db.php";

$search = trim($_GET["q"] ?? "");

if ($search !== "") {
    // READ with a filter - still a prepared statement
    $stmt = $conn->prepare(
        "SELECT * FROM manuscripts
          WHERE title LIKE ? OR accession LIKE ? OR script LIKE ?
          ORDER BY created_at DESC"
    );
    $like = "%" . $search . "%";
    $stmt->bind_param("sss", $like, $like, $like);
} else {
    $stmt = $conn->prepare("SELECT * FROM manuscripts ORDER BY created_at DESC");
}

$stmt->execute();
$result = $stmt->get_result();
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>LipiSutra Archive</title>
<style>
  body  { font-family:Georgia,serif; background:#f5ede0; color:#1a0800;
padding:24px; }
  table { border-collapse:collapse; width:100%; background:#fff; }
  th,td { border:1px solid #8B4513; padding:8px; font-size:14px; text-align:left; }
  th    { background:#ede0cc; }
  .ok   { color:#1e6b33; font-weight:bold; }
  .pend { color:#a15c00; font-weight:bold; }
  a.btn { padding:4px 10px; border:1px solid #8B4513; border-radius:4px;
          text-decoration:none; color:#6B2D0E; font-size:13px; }
</style></head>
<body>

  <h1>LipiSutra Manuscript Archive</h1>

  <?php if (isset($_GET["msg"])): ?>
    <p class="ok"><?= htmlspecialchars($_GET["msg"]) ?></p>
  <?php endif; ?>

  <form method="get">
    <input type="text" name="q" placeholder="Search title, accession or script"
```

```
           value="<?= htmlspecialchars($search) ?>" size="40">
    <button type="submit">Search</button>
    <a class="btn" href="index.php">Reset</a>
    <a class="btn" href="create.php">+ Add Manuscript</a>
  </form>

  <p><?= $result->num_rows ?> record(s) found.</p>

  <table>
    <tr>
      <th>ID</th><th>Accession</th><th>Title</th><th>Script</th>
      <th>Year</th><th>Confidence</th><th>Status</th><th>Actions</th>
    </tr>

    <?php if ($result->num_rows === 0): ?>
      <tr><td colspan="8">No manuscripts match the search.</td></tr>
    <?php endif; ?>

    <?php while ($row = $result->fetch_assoc()): ?>
      <tr>
        <td><?= (int)$row["id"] ?></td>
        <td><?= htmlspecialchars($row["accession"]) ?></td>
        <td><?= htmlspecialchars($row["title"]) ?></td>
        <td><?= htmlspecialchars($row["script"]) ?></td>
        <td><?= $row["era_year"] < 0
                 ? abs($row["era_year"]) . " BCE"
                 : $row["era_year"] . " CE" ?></td>
        <td><?= number_format($row["confidence"], 2) ?>%</td>
        <td class="<?= $row["status"] === "verified" ? "ok" : "pend" ?>">
          <?= htmlspecialchars($row["status"]) ?>
        </td>
        <td>
          <a class="btn" href="update.php?id=<?= (int)$row["id"] ?>">Edit</a>
          <a class="btn" href="delete.php?id=<?= (int)$row["id"] ?>"
             onclick="return confirm('Delete <?= htmlspecialchars($row["accession"]) ?
>?');">
             Delete</a>
        </td>
      </tr>
    <?php endwhile; ?>
  </table>

<?php
$stmt->close();
$conn->close();
?>
</body>
</html>
```

### File 4: create.php - CREATE

```
<?php
require "db.php";

$errors = [];

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $accession  = trim($_POST["accession"]  ?? "");
    $title      = trim($_POST["title"]      ?? "");
    $script     = trim($_POST["script"]     ?? "");
    $era_year   = trim($_POST["era_year"]   ?? "");
    $confidence = trim($_POST["confidence"] ?? "");
    $transcript = trim($_POST["transcript"] ?? "");

    // ---------- server-side validation ----------
    if (!preg_match('/^LS-\d{4}$/', $accession))      $errors[] = "Accession must be
LS-0147.";
```

```
    if (mb_strlen($title) < 5)                          $errors[] = "Title is too
short.";
    if (!in_array($script, ["Modi","Devanagari","Brahmi","Sharada"], true))
                                                        $errors[] = "Choose a valid
script.";
    if (filter_var($era_year, FILTER_VALIDATE_INT,
        ["options" => ["min_range" => -500, "max_range" => 2026]]) === false)
                                                        $errors[] = "Year must be -
500..2026.";
    if (!is_numeric($confidence) || $confidence < 0 || $confidence > 100)
                                                        $errors[] = "Confidence must be
0..100.";

    if (empty($errors)) {
        // ---------- CREATE : prepared INSERT ----------
        $stmt = $conn->prepare(
            "INSERT INTO manuscripts
               (accession, title, script, era_year, confidence, transcript)
             VALUES (?, ?, ?, ?, ?, ?)"
        );
        // s=string  i=integer  d=double
        $stmt->bind_param("sssids", $accession, $title, $script,
                                    $era_year, $confidence, $transcript);

        if ($stmt->execute()) {
            $newId = $conn->insert_id;
            $stmt->close();
            $conn->close();
            header("Location: index.php?msg=" .
                   urlencode("Manuscript created with id $newId."));
            exit;
        }

        $errors[] = ($conn->errno === 1062)
            ? "That accession number already exists."
            : "Insert failed: " . $stmt->error;
        $stmt->close();
    }
}
?>
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><title>Add Manuscript</title></head>
<body style="font-family:Georgia,serif;background:#f5ede0;padding:24px">

  <h1>Add a Manuscript</h1>
  <?php foreach ($errors as $e): ?>
    <p style="color:#b00020"><?= htmlspecialchars($e) ?></p>
  <?php endforeach; ?>

  <form method="post">
    Accession <input name="accession" placeholder="LS-0147"
                     value="<?= htmlspecialchars($_POST["accession"] ?? "") ?
>"><br><br>
    Title     <input name="title" size="40"
                     value="<?= htmlspecialchars($_POST["title"] ?? "") ?>"><br><br>
    Script
    <select name="script">
      <?php foreach (["Modi","Devanagari","Brahmi","Sharada"] as $s): ?>
        <option <?= (($_POST["script"] ?? "") === $s) ? "selected" : "" ?>><?= $s ?
></option>
      <?php endforeach; ?>
    </select><br><br>
    Year       <input name="era_year" value="<?=
htmlspecialchars($_POST["era_year"] ?? "") ?>"><br><br>
    Confidence <input name="confidence" value="<?=
htmlspecialchars($_POST["confidence"] ?? "") ?>"><br><br>
    Transcript <textarea name="transcript" rows="4" cols="50"><?=
                  htmlspecialchars($_POST["transcript"] ?? "") ?></textarea><br><br>

    <button type="submit">Save</button>
```

```
    <a href="index.php">Cancel</a>
  </form>
</body></html>
```

### File 5: update.php - UPDATE

```
<?php
require "db.php";

$id = filter_input(INPUT_GET, "id", FILTER_VALIDATE_INT)
   ?: filter_input(INPUT_POST, "id", FILTER_VALIDATE_INT);

if (!$id) { header("Location: index.php?msg=" . urlencode("Invalid id.")); exit; }

$errors = [];

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $title      = trim($_POST["title"]      ?? "");
    $script     = trim($_POST["script"]     ?? "");
    $era_year   = trim($_POST["era_year"]   ?? "");
    $confidence = trim($_POST["confidence"] ?? "");
    $status     = trim($_POST["status"]     ?? "");
    $transcript = trim($_POST["transcript"] ?? "");

    if (mb_strlen($title) < 5) $errors[] = "Title is too short.";
    if (!in_array($status, ["pending","verified"], true)) $errors[] = "Invalid
status.";

    if (empty($errors)) {
        // ---------- UPDATE : prepared statement with a WHERE clause ----------
        $stmt = $conn->prepare(
            "UPDATE manuscripts
                SET title = ?, script = ?, era_year = ?, confidence = ?,
                    status = ?, transcript = ?
              WHERE id = ?"
        );
        $stmt->bind_param("ssidssi", $title, $script, $era_year,
                                     $confidence, $status, $transcript, $id);
        $stmt->execute();
        $changed = $stmt->affected_rows;      // 0 if nothing actually differed
        $stmt->close();
        $conn->close();

        header("Location: index.php?msg=" .
               urlencode($changed > 0 ? "Record $id updated."
                                      : "No change was needed for record $id."));
        exit;
    }
}

// ---------- READ the current row to pre-fill the form ----------
$stmt = $conn->prepare("SELECT * FROM manuscripts WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$row = $stmt->get_result()->fetch_assoc();
$stmt->close();

if (!$row) { header("Location: index.php?msg=" . urlencode("Record not found."));
exit; }
?>
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><title>Edit Manuscript</title></head>
<body style="font-family:Georgia,serif;background:#f5ede0;padding:24px">

  <h1>Edit <?= htmlspecialchars($row["accession"]) ?></h1>
  <?php foreach ($errors as $e): ?>
```

```
    <p style="color:#b00020"><?= htmlspecialchars($e) ?></p>
  <?php endforeach; ?>

  <form method="post">
    <input type="hidden" name="id" value="<?= (int)$row["id"] ?>">

    Title <input name="title" size="40"
                 value="<?= htmlspecialchars($row["title"]) ?>"><br><br>
    Script
    <select name="script">
      <?php foreach (["Modi","Devanagari","Brahmi","Sharada"] as $s): ?>
        <option <?= $row["script"] === $s ? "selected" : "" ?>><?= $s ?></option>
      <?php endforeach; ?>
    </select><br><br>
    Year       <input name="era_year"   value="<?= (int)$row["era_year"] ?>"><br><br>
    Confidence <input name="confidence" value="<?=
htmlspecialchars($row["confidence"]) ?>"><br><br>
    Status
    <select name="status">
      <option <?= $row["status"] === "pending"  ? "selected" : "" ?>>pending</option>
      <option <?= $row["status"] === "verified" ? "selected" : "" ?>>verified</option>
    </select><br><br>
    Transcript <textarea name="transcript" rows="4" cols="50"><?=
                 htmlspecialchars($row["transcript"]) ?></textarea><br><br>

    <button type="submit">Update</button>
    <a href="index.php">Cancel</a>
  </form>
</body></html>
```

### File 6: delete.php - DELETE

```
<?php
require "db.php";

// FILTER_VALIDATE_INT rejects "1 OR 1=1" before it ever reaches the database
$id = filter_input(INPUT_GET, "id", FILTER_VALIDATE_INT);

if (!$id) {
    header("Location: index.php?msg=" . urlencode("Invalid id supplied."));
    exit;
}

// ---------- DELETE : prepared statement with a bound integer ----------
$stmt = $conn->prepare("DELETE FROM manuscripts WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();

$removed = $stmt->affected_rows;
$stmt->close();
$conn->close();

header("Location: index.php?msg=" .
       urlencode($removed > 0 ? "Record $id deleted."
                              : "No record with id $id was found."));
exit;
```

### Project evidence - src/services/firebase.js (the same CRUD cycle in NoSQL)

```
// CREATE
export async function saveDocument(geminiResult, role, hash) {
  await setDoc(doc(db, "documents", hash), {
    view_count: 1, status: 'pending', priority_score: 80,
    ai_output: geminiResult, historian_output: null,
    timestamp: serverTimestamp()
  });
}
```

```
// READ one
export async function checkExistingHash(hash) {
  const docSnap = await getDoc(doc(db, "documents", hash));
  if (docSnap.exists()) {
    const data = docSnap.data();
    if (data.status === 'verified') return data.historian_output;
    if (data.status === 'pending')  return data.ai_output;
  }
  return null;
}

// READ many, ordered and limited  (SELECT ... ORDER BY ... LIMIT)
export async function getRecentDocuments(max = 20) {
  const q = query(collection(db, "documents"), orderBy("timestamp", "desc"),
limit(max));
  const snapshot = await getDocs(q);
  return snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
}

// CREATE with an auto-generated key  (INSERT with AUTO_INCREMENT)
export async function saveCorrection(correction) {
  return await addDoc(collection(db, "corrections"), {
    ...correction, savedByRole: "historian", timestamp: serverTimestamp()
  });
}

// UPDATE  (UPDATE ... SET searchCount = searchCount + 1 WHERE id = ?)
updateDoc(doc(db, "documents", hash), { searchCount: increment(1) });
```

### Steps to run

```
1. Start Apache and MySQL from the XAMPP control panel.
2. Open http://localhost/phpmyadmin, go to the SQL tab and run lipisutra.sql.
3. Copy db.php, index.php, create.php, update.php and delete.php into
   C:\xampp\htdocs\lipisutra_crud\
4. Open http://localhost/lipisutra_crud/index.php
5. Exercise each operation: Add, Search, Edit and Delete.
```

---

**Output:**

- index.php lists the three seeded manuscripts with the year rendered as "1712 CE" and "230 BCE" and the status coloured green for verified and amber for pending.
- Typing "modi" in the search box returns only the Modi Land Grant row and the count line reads "1 record(s) found."
- Adding LS-0104 / "Sharada Folio" redirects back to the list with the green message "Manuscript created with id 4." and the new row at the top.
- Re-submitting LS-0104 is rejected with "That accession number already exists.", which is the UNIQUE constraint being reported through MySQL error 1062.
- Submitting an accession of LS-14 or a confidence of 150 is rejected by the server-side validation before any query runs.
- Editing record 4 to status "verified" redirects with "Record 4 updated."; pressing Update again without changing anything reports "No change was needed for record 4.", because affected_rows is 0.
- Clicking Delete raises the JavaScript confirm dialog; confirming removes the row and reports "Record 4 deleted."
- Manually requesting delete.php?id=1%20OR%201=1 is rejected by FILTER_VALIDATE_INT with "Invalid id supplied." and no row is removed, demonstrating the injection defence.

**Conclusion:**

In this assignment, we developed a PHP-MySQL application performing database connectivity and all four CRUD operations. We created the schema with SQL DDL, connected using MySQLi with the utf8mb4 character set so that Devanagari transcripts are stored correctly, and implemented Create, Read with a search filter, Update and Delete. Every query was written as a prepared statement with bound parameters, which we verified prevents SQL injection. We also compared this with the LipiSutra course project, whose src/services/firebase.js performs the same CRUD cycle against a NoSQL document store.

---

**References:**


9. PHP Manual - Language Reference, https://www.php.net/manual/en/langref.php
10. PHP Manual - Session Handling, https://www.php.net/manual/en/book.session.php
11. W3Schools - PHP Tutorial, https://www.w3schools.com/php/
12. PHP Manual - MySQLi, https://www.php.net/manual/en/book.mysqli.php
13. PHP Manual - Prepared Statements, https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
14. MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/refman/8.0/en/